\

HKBK COLLEGE OF ENGINEERING
(Affiliated to VTU, Belgaum and Approved by AICTE)
BANGALORE -560045
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Generative AI

BAIL657C

As per VTU 22 Scheme

(Effective from the academic year of 2024 -2025)

VI Semester

LABORATORY MANUAL

Prepared By
Ms. Deepa K

Verified By

DQAC	**HOD**

\

HKBK COLLEGE OF ENGINEERING

(Affiliated to VTU, Belgaum and Approved by AICTE)
BANGALORE -560045
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Vision and Mission of the Institution

Vision

To empower students through wholesome education and enable the students to develop into highly qualified and trained professionals with ethics and emerge as responsible citizen with broad outlook to build a vibrant nation.

Mission

- To achieve academic excellence in science, engineering and technology through dedication to duty, innovation in teaching and faith in human values.
- To enable our students to develop into outstanding professional with high ethical standards to face the challenges of 21st century.
- To provide educational opportunities to the deprived and weaker section of the society to uplift their socioeconomic status.

Vision and Mission of the AI/ML Department

Vision

To advance the intellectual capacity of the nation and the international community by imparting knowledge to graduates who are globally recognized as innovators, entrepreneur and competent professionals.

Mission

- To provide excellent technical knowledge and computing skills to make the graduates globally competitive with professional ethics.
- To involve in research activities and be committed to lifelong learning to make positive contributions to the society

\

HKBK COLLEGE OF ENGINEERING

(Affiliated to VTU, Belgaum and Approved by AICTE)
BANGALORE -560045

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Programme Outcomes	
PO1	**Engineering Knowledge:** Apply knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem Analysis:** Identify, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences
PO3	**Design/Development of Solutions**: Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.
PO4	**Conduct investigations of complex problems:** Using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.
PO5	**Modern Tool Usage**: Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering activities with an under- standing of the limitations.
PO6	**The Engineer and Society:** Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
PO7	**Environment and Sustainability**: Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.
PO8	**Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	**Individual and Team Work**: Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
PO10	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions**.**
PO11	**Project Management and Finance**: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-long Learning**: Recognize the need for and have the preparation and ability to engage in independent and life- long learning in the broadest context of technological change.

\

HKBK COLLEGE OF ENGINEERING

(Affiliated to VTU, Belgaum and Approved by AICTE)
BANGALORE -560045
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Programme Specific Outcomes	
PSO1	**Design thinking and problem- solving skill:** Understand, formulate and solve interdisciplinary problems with AI tools.
PSO2	**Professional Skill:** A skill to apply the AI and ML appropriate techniques and contribute to society with innovations, research and entrepreneurships.

Programme Educational Objectives

PEO-1	Graduates will possess fundamental knowledge of engineering and will pursue successful career as Software Professionals, AI Developers, Data Analyst, and Data Scientists.
PEO-2	Graduates will demonstrate lifelong learning in the field of Artificial Intelligence and Machine Learning
PEO-3	To equip students in the chosen field of engineering and related fields to enable him to work in multidisciplinary teams.

INDEX

Sl. No	Contents	Page No
1	Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.	12
2	Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.	15
3	Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.	17
4	Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.	19
5	Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: • Takes a seed word. • Generates similar words. • Constructs a short paragraph using these words.	21
6	Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application. Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.	23
7	Summarize long texts using a pre-trained summarization model using Hugging Face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.	24
8	Install langchain, cohere (for key), langchain-community. Get the API key (By logging into Cohere and obtaining the Cohere key). Load a text document from your Google Drive. Create a prompt template to display the output in a particular manner.	25
9	Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: • The founder of the Institution. • When it was founded. • The current branches in the institution. • How many employees are working in it. • A brief 4-line summary of the institution.	27

10	Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.	29

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together

Continuous Internal Evaluation (CIE):

CIE marks for the practical course are 50 Marks. The split-up of CIE marks for record/ journal and test are in the ratio 60:40.

● Each experiment is to be evaluated for conduction with an observation sheet and record write-up. Rubrics for the evaluation of the journal/write-up for hardware/software experiments are designed by the faculty who is handling the laboratory session and are made known to students at the beginning of the practical session.

● Record should contain all the specified experiments in the syllabus and each experiment write-up will be evaluated for 10 marks.

● Total marks scored by the students are scaled down to 30 marks (60% of maximum marks).

● Weightage to be given for neatness and submission of record/write-up on time.

● Department shall conduct a test of 100 marks after the completion of all the experiments listed in the syllabus.

● In a test, test write-up, conduction of experiment, acceptable result, and procedural knowledge will carry a weightage of 60% and the rest 40% for viva-voce.

● The suitable rubrics can be designed to evaluate each student's performance and learning ability.

● The marks scored shall be scaled down to 20 marks (40% of the maximum marks). The Sum of scaled-down marks scored in the report write-up/journal and marks of a test is the total CIE marks scored by the student.

Semester End Evaluation (SEE):

● SEE marks for the practical course are 50 Marks.

● SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the Head of the Institute.

● The examination schedule and names of examiners are informed to the university before the conduction of the examination. These practical examinations are to be conducted between the schedule mentioned in the academic calendar of the University. Template for Practical Course and if AEC is a practical Course Annexure-V

● All laboratory experiments are to be included for practical examination.

● (Rubrics) Breakup of marks and the instructions printed on the cover page of the answer script to be strictly adhered to by the examiners. OR based on the course requirement evaluation rubrics shall be decided jointly by examiners.

● Students can pick one question (experiment) from the questions lot prepared by the examiners jointly.

● Evaluation of test write-up/ conduction procedure and result/viva will be conducted jointly by examiners.

● General rubrics suggested for SEE are mentioned here, writeup-20%, Conduction procedure

and result in -60%, Viva-voce 20% of maximum marks. SEE for practical shall be evaluated for 100 marks and scored marks shall be scaled down to 50 marks (however, based on course type, rubrics shall be decided by the examiners) part are to be made zero. The minimum duration of SEE is 02 hours.

● Change of experiment is allowed only once and 15% of Marks allotted to the procedure part are to be made zero.

The minimum duration of SEE is 02 hours.

Course objectives:

CO1	**Understand the principles and concepts behind generative AI models.**
CO2	**Explain the knowledge gained to implement generative models using Prompt design frameworks.**
CO3	**Apply various Generative AI applications for increasing productivity.**
CO4	**Develop Large Language Model-based Apps.**

Lab Assessment Rubrics for CIE

Sl No	Description	Marks
1	***Continuous Evaluation***	***30***
	a. Observation write up and punctuality	5
	b. Conduction of experiment and output	10
	c. Viva voice	5
	d. Record write up	10
2	***Internal Test***	***20***

What is a GENERATIVE AI?

Generative AI is a type of Artificial Intelligence that can create new content such as text, images, audio, video, code, and designs by learning patterns from existing data.

It uses advanced machine learning models, especially Large Language Models (LLMs) and deep learning techniques, to generate human-like outputs based on user input (prompts).

Examples of Generative AI

- Text generation: ChatGPT writing essays, emails, or code.
- Image generation: AI creating artwork or realistic images from text descriptions.
- Music generation: AI composing songs or background music.
- Video generation: AI creating animations or video clips.
- Code generation: AI writing or debugging programs.

How Generative AI Works

1. The model is trained on large amounts of data.
2. It learns patterns, relationships, and structures in the data.
3. When given a prompt, it predicts and generates relevant content.

Features of GENERATIVE AI

Features of Generative AI

1. **Content Generation**

   Generates new content such as text, images, audio, video, and code.
2. **Human-like Responses**

   Produces natural and conversational outputs similar to human communication.
3. **Learning from Large Data**

   Trains on huge datasets to understand patterns, language, and relationships.
4. **Context Awareness**

   Understands prompts and maintains context during conversations or tasks.
5. **Creativity and Innovation**

   Creates stories, designs, music, artwork, and unique ideas.
6. **Automation**

   Automates repetitive tasks like report writing, coding, summarization, and email drafting.
7. **Multimodal Capability**

   Can process and generate multiple data types such as text, images, and audio together.

8. **Personalization**

   Generates customized responses based on user input and preferences.

9. **Language Understanding**

   Supports translation, summarization, question answering, and sentiment analysis.

10. **Code Generation and Debugging**

    Assists programmers by generating, explaining, and debugging code.

11. **Continuous Improvement**

    Improves performance through training updates and fine-tuning.

12. **Scalability**

    Handles large-scale tasks and serves many users simultaneously.

13. **Interactive Communication**

    Enables real-time interaction through chatbots and virtual assistants.

14. **Knowledge Retrieval**

    Retrieves and combines information from various sources to provide meaningful outputs.

15. **Adaptive Responses**

    Adjusts outputs according to different prompts, styles, and domains.

Experiment-1

Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.

```
import nltk
from nltk.corpus import stopwords
from nltk import FreqDist

# download stopwords
nltk.download('stopwords')

# sample corpus
corpus = [
    'king is a strong man',
    'queen is a wise woman',
    'boy is a young man',
    'girl is a young woman',
    'prince is a young king',
    'prince will be strong',
    'prince is young',
    'man is strong',
    'woman is pretty',
    'prince is a boy',
    'prince will be king',
    'princess is a girl',
    'princess will be queen'
]

# tokenize sentences
sl = [s.split() for s in corpus]
print("\nTokenized statements:")
print(sl)

# remove stopwords
sw = set(stopwords.words('english'))
doc = [[w for w in s if w.lower() not in sw] for s in sl]
print("\nDocuments after removing stopwords:")
print(doc)

# create frequency distribution
v = FreqDist([w for d in doc for w in d])
print("\nVocabulary (word frequencies):")
print(dict(v))

# get frequencies of specific words
vk = v['king']
vm = v['man']
vw = v['woman']
vq = v['queen']

print(f"\nFrequency of 'king': {vk}")
print(f"Frequency of 'man': {vm}")
print(f"Frequency of 'woman': {vw}")
print(f"Frequency of 'queen': {vq}")
```

```
# vector arithmetic
sv = vk + vm
dv = vk - vm

print(f"\nSum Vector (king + man): {sv}")
print(f"Difference Vector (king - man): {dv}")

# cosine similarity (simplified)
simi = (vk * vq) / ((vk ** 0.5) * (vq ** 0.5))
print(f"\nCosine Similarity between 'king' and 'queen': {simi}")

# most frequent word
ms = sorted(v.items(), key=lambda item: item[1], reverse=True)
print("\nMost similar word to 'king' (by frequency):")
print(ms)

# analogy: king - man + woman
ana_vec = vk - vm + vw

def dis(item):
    return abs(item[1] - ana_vec)

mse = sorted(v.items(), key=dis)[1]
print(f"\nAnalogy Result (king - man + woman = ?): {mse}")
```

Output:

```
Tokenized statements:
[['king', 'is', 'a', 'strong', 'man'], ['queen', 'is', 'a', 'wise', 'woman'], ['boy', 'is', 'a', 'young', 'man'], ['girl', 'is', 'a', 'young', 'woman'], ['prince', 'is', 'a', 'young', 'king'], ['prince', 'will', 'be', 'strong'], ['prince', 'is', 'young'], ['man', 'is', 'strong'], ['woman', 'is', 'pretty'], ['prince', 'is', 'a', 'boy'], ['prince', 'will', 'be', 'king'], ['princess', 'is', 'a', 'girl'], ['princess', 'will', 'be', 'queen']]

Documents after removing stopwords:
[['king', 'strong', 'man'], ['queen', 'wise', 'woman'], ['boy', 'young', 'man'], ['girl', 'young', 'woman'], ['prince', 'young', 'king'], ['prince', 'strong'], ['prince', 'young'], ['man', 'strong'], ['woman', 'pretty'], ['prince', 'boy'], ['prince', 'king'], ['princess', 'girl'], ['princess', 'queen']]

Vocabulary (word frequencies):
{'king': 3, 'strong': 3, 'man': 3, 'queen': 2, 'wise': 1, 'woman': 3, 'boy': 2, 'young': 4, 'girl': 2, 'prince': 5, 'pretty': 1, 'princess': 2}

Frequency of 'king': 3
Frequency of 'man': 3
Frequency of 'woman': 3
Frequency of 'queen': 2

Sum Vector (king + man): 6
Difference Vector (king - man): 0

Cosine Similarity between 'king' and 'queen': 2.449489742783178

Most similar word to 'king' (by frequency):
```

[('prince', 5), ('young', 4), ('king', 3), ('strong', 3), ('man', 3), ('woman', 3), ('queen', 2), ('boy', 2), ('girl', 2), ('princess', 2), ('wise', 1), ('pretty', 1)]

Analogy Result (king - man + woman = ?): ('strong', 3)

Experiment-2

Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity

# Sample sports sentences
corpus = [
    "basketball is a sport played with a ball",
    "soccer is played by two teams on a field",
    "football involves a lot of physical contact",
    "athletes train hard to improve their performance",
    "coaching is an important part of every sport",
    "basketball players need good coordination",
    "a team consists of players and a coach",
    "training and exercise are important for health",
    "soccer players score goals to win games",
    "football teams compete in leagues and tournaments"
]

# Step 1: Convert text to word vectors
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus).toarray()
words = vectorizer.get_feature_names_out()

# Step 2: Apply PCA to reduce to 2D
pca = PCA(n_components=2)
word_vectors_2d = pca.fit_transform(X.T)

# Step 3: Plot the words
plt.figure(figsize=(8, 8))
plt.scatter(word_vectors_2d[:, 0], word_vectors_2d[:, 1])

for i, word in enumerate(words):
    plt.text(word_vectors_2d[i, 0], word_vectors_2d[i, 1], word, fontsize=10)

plt.title("2D Visualization of Sports Word Embeddings (PCA)")
plt.show()

# Step 4: Find 5 similar words
def find_similar_words(target_word, words, vectors, top_n=5):
    if target_word not in words:
        print("Word not found!")
        return

    idx = list(words).index(target_word)
    target_vector = vectors[idx].reshape(1, -1)
```

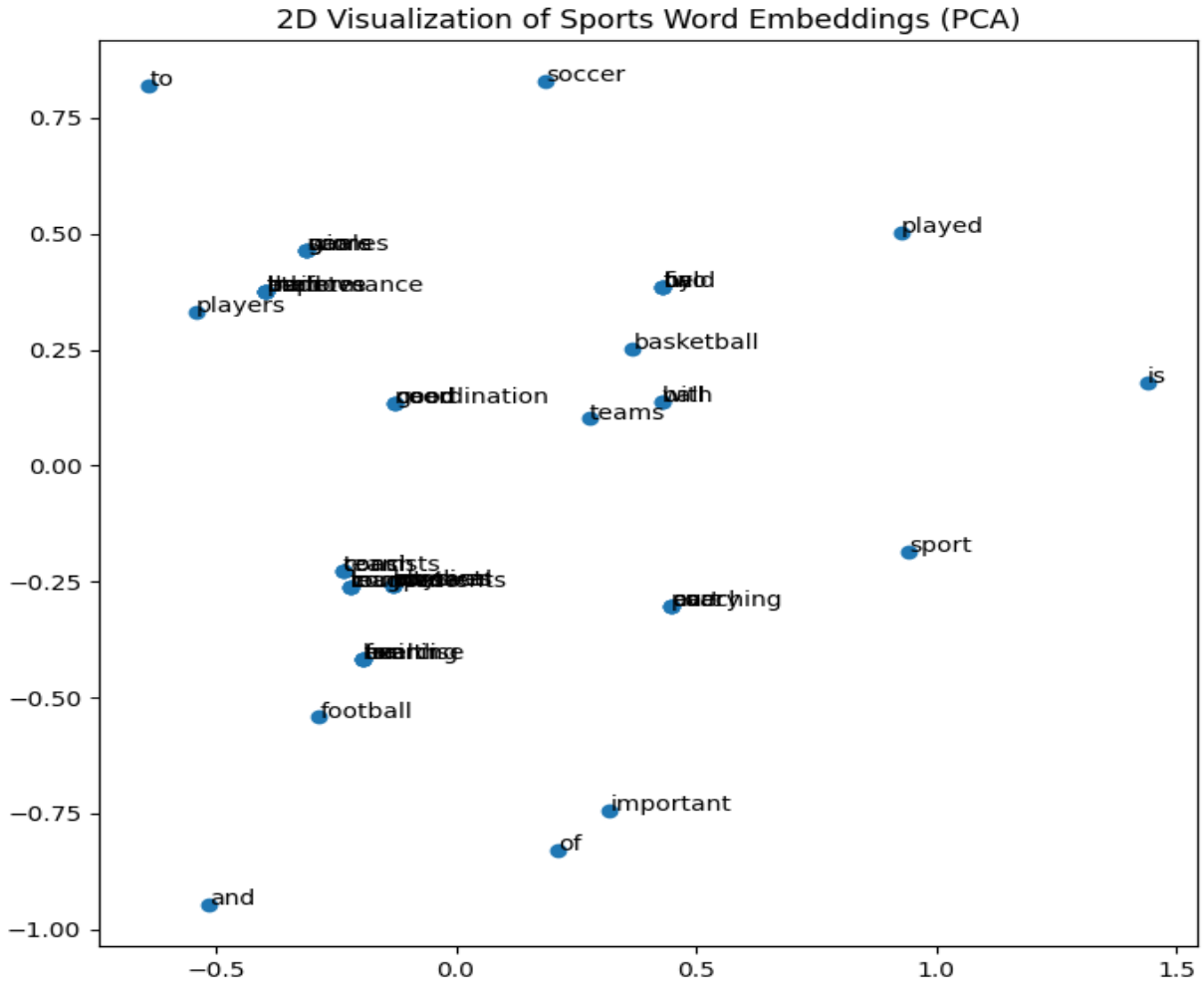
```
    similarities = cosine_similarity(target_vector, vectors)[0]

    # Get top similar words (excluding itself)
    similar_indices = similarities.argsort()[::-1][1:top_n+1]

    print(f"\nTop {top_n} words similar to '{target_word}':")
    for i in similar_indices:
        print(words[i])

# Example usage
find_similar_words("soccer", words, X.T, top_n=5)
```

Output:



Top 5 words similar to 'soccer':
games
win
on
goals
score

Experiment-3

Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.

Cmd: pip install gensim matplotlib scikit-learn numpy

```
import numpy as np
import matplotlib.pyplot as plt
from gensim.models import Word2Vec
from sklearn.decomposition import PCA

# Legal domain corpus
sentences = [
    "The defendant is guilty of the crime",
    "The plaintiff filed a lawsuit against the defendant",
    "The court ruled in favour of the defendant",
    "The legal proceedings were delayed by the judge",
    "The lawyer presented evidence in court",
    "A judge will issue a verdict after the trial",
    "The defendant has the right to remain silent",
    "A contract is a legally binding agreement",
    "The witness was cross examined by the attorney",
    "The legal team will appeal the decision"
]

# Step 1: Tokenization
tokenized = [s.lower().split() for s in sentences]

# Step 2: Train Word2Vec model
model = Word2Vec(tokenized, vector_size=100, window=5, min_count=1)

# Step 3: Show vocabulary
words = list(model.wv.index_to_key)
print("Vocabulary:", words)

# Step 4: Find similar words
target_word = "defendant"
print(f"\nWords similar to '{target_word}':")
similar = model.wv.most_similar(target_word, topn=5)

for word, score in similar:
    print(word, "->", round(score, 3))

# Step 5: Prepare word vectors for plotting
word_vectors = model.wv[words]

# Step 6: Reduce dimensions to 2D using PCA
pca = PCA(n_components=2)
reduced_vectors = pca.fit_transform(word_vectors)

# Step 7: Scatter plot
plt.figure(figsize=(8, 6))
```

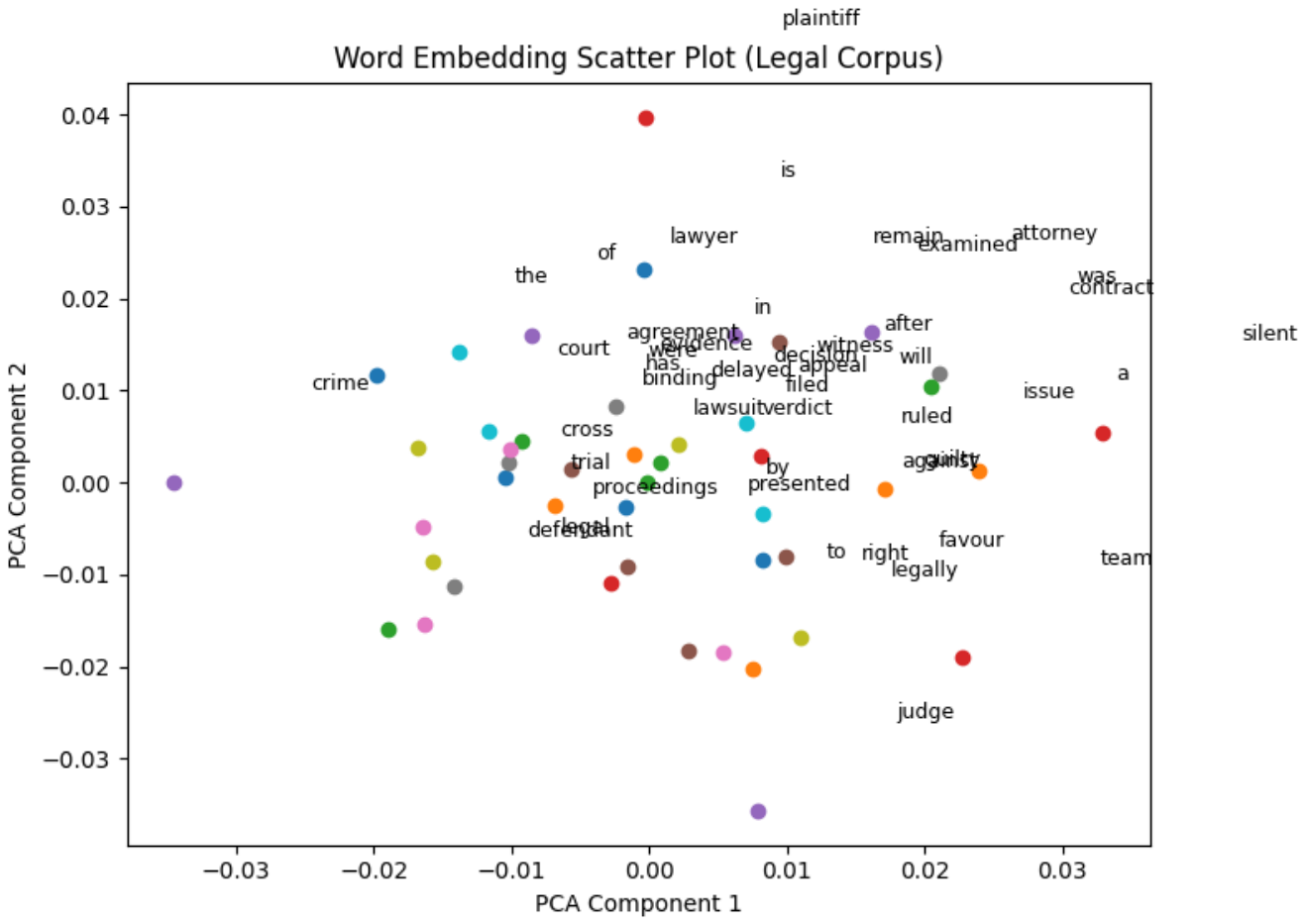
```
for i, word in enumerate(words):
    x, y = reduced_vectors[i]
    plt.scatter(x, y)
    plt.text(x + 0.01, y + 0.01, word, fontsize=9)

plt.title("Word Embedding Scatter Plot (Legal Corpus)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

Output:

Vocabulary: ['the', 'a', 'defendant', 'will', 'judge', 'by', 'legal', 'in', 'court', 'of', 'is', 'decision', 'appeal', 'team', 'attorney', 'examined', 'cross', 'was', 'witness', 'agreement', 'binding', 'legally', 'contract', 'silent', 'remain', 'to', 'right', 'has', 'trial', 'after', 'verdict', 'issue', 'evidence', 'presented', 'lawyer', 'delayed', 'were', 'proceedings', 'favour', 'ruled', 'against', 'lawsuit', 'filed', 'plaintiff', 'crime', 'guilty']

Words similar to 'defendant':
lawsuit -> 0.212
court -> 0.199
decision -> 0.173
after -> 0.171
judge -> 0.17



Experiment-4

Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

Cmd: pip install gensim nltk transformers torch
Cmd: pip install nltk

```
from gensim.models import Word2Vec
from transformers import pipeline

# Step 1: Legal domain corpus
corpus = [
    "The defendant argued the case in court",
    "The judge gave the final verdict",
    "The lawyer presented evidence",
    "The jury listened to the trial",
    "Justice requires fairness and law",
    "The court examines the evidence carefully"
]

# Step 2: Tokenize
tokenized_corpus = [sentence.lower().split() for sentence in corpus]

# Step 3: Train Word2Vec
model = Word2Vec(tokenized_corpus, vector_size=50, window=3, min_count=1)

# Step 4: Get similar words
def get_similar_words(word, topn=5):
    if word in model.wv:
        return [w for w, _ in model.wv.most_similar(word, topn=topn)]
    return []

# Step 5: Enrich prompt
def enrich_prompt(original_prompt, similar_words):
    return f"{original_prompt} Include terms like {', '.join(similar_words)}."

# Step 6: Real AI generator
generator = pipeline("text-generation", model="distilgpt2")

def generate_response(prompt):
    output = generator(prompt, max_new_tokens=50, do_sample=True)
    return output[0]["generated_text"]

# Step 7: Original prompt
original_prompt = "Explain the role of a defendant in a court case."

# Step 8: Process
```

```
similar_words = get_similar_words("defendant")
enriched_prompt = enrich_prompt(original_prompt, similar_words)

# Step 9: Generate responses
original_response = generate_response(original_prompt)
enriched_response = generate_response(enriched_prompt)

# Step 10: Output
print("Most similar words to 'defendant':", similar_words)

print("\n--- Original Prompt Response ---\n")
print(original_response)

print("\n--- Enriched Prompt Response ---\n")
print(enriched_response)
```

Output:

```
Most similar words to 'defendant': ['presented', 'to', 'law', 'the', 'court']

--- Original Prompt Response ---

Explain the role of a defendant in a court case.

If a defendant has no more than one person in the courtroom, they cannot be charged with a crime.
If a defendant has no more than one person in the courtroom, they cannot be charged with a crime.
If a

--- Enriched Prompt Response ---

Explain the role of a defendant in a court case. Include terms like presented, to, law, the, court.
```

Experiment-5

Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that:

- **Takes a seed word.**
- **Generates similar words.**
- **Constructs a short paragraph using these words.**

```
import random
import gensim.downloader as api

# Load pre-trained word embedding model
model = api.load("glove-wiki-gigaword-50")

def get_similar_words(seed_word, top_n=5):
    """Retrieve similar words for the given seed word."""
    try:
        similar_words = [word for word, _ in model.most_similar(seed_word, topn=top_n)]
        return similar_words
    except KeyError:
        return []

def create_paragraph(seed_word):
    """Generate a short paragraph using the seed word and its similar words."""
    similar_words = get_similar_words(seed_word)

    if not similar_words:
        return f"Could not find similar words for '{seed_word}'. Try another word!"

    paragraph = (
        f"Once upon a time, a {seed_word} embarked on a journey. Along the way, it
encountered "
        f"a {random.choice(similar_words)}, which led it to a hidden
{random.choice(similar_words)}. "
        f"Despite the challenges, it found {random.choice(similar_words)} and embraced the "
        f"adventure with {random.choice(similar_words)}. In the end, the journey was a tale of
"
        f"{random.choice(similar_words)} and discovery."
    )

    return paragraph

seed_word = input("Enter a seed word: ").strip().lower()
print("\nGenerated Story:\n")
print(create_paragraph(seed_word))
```

Output:

Enter a seed word: forest

Generated Story:

Once upon a time, a forest embarked on a journey. Along the way, it encountered a wildlife, which led it to a hidden woodland. Despite the challenges, it found woodland and embraced the adventure with wildlife. In the end, the journey was a tale of woodlands and discovery.

Experiment-6

Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application. Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.

```
from transformers import pipeline
sentiment_analyzer=pipeline("sentiment-analysis")

def analyze_sentiment(text):
        """Analyze sentiment of the input text using Hugging Face pipeline"""
        result=sentiment_analyzer(text)
        label=result[0]['label']
        score=result[0]['score']

        return f"Sentiment: {label} (Confidence: {score:.2f})"

while True:
        user_input=input("Enter a sentence for sentiment analysis( or 'exit' to quit):").strip()
        if user_input.lower()=='exit':
                break
        print(analyze_sentiment(user_input))
```

Output:

```
Enter a sentence for sentiment analysis( or 'exit' to quit): I love this product! It's amazing
Sentiment: POSITIVE (Confidence: 1.00)
Enter a sentence for sentiment analysis( or 'exit' to quit): This is the worst experience ever
Sentiment: NEGATIVE (Confidence: 1.00)
Enter a sentence for sentiment analysis( or 'exit' to quit): The service was okay, nothing special
Sentiment: NEGATIVE (Confidence: 0.99)
```

Experiment-7

Summarize long texts using a pre-trained summarization model using Hugging Face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

```
from transformers import BartTokenizer, BartForConditionalGeneration

# Load tokenizer and model
tokenizer = BartTokenizer.from_pretrained("facebook/bart-large-cnn")
model = BartForConditionalGeneration.from_pretrained("facebook/bart-large-cnn")

# Take input text
text = input("Enter the passage to summarize:\n")

# Tokenize input (max 1024 tokens for BART)
inputs = tokenizer(text, max_length=1024, return_tensors="pt", truncation=True)

# Generate summary
summary_ids = model.generate(
   inputs["input_ids"],
   max_length=150,
   min_length=40,
   length_penalty=2.0,
   num_beams=4,
   early_stopping=True
)

# Decode summary
summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

print("\nSummarized Text:")
print(summary)
```

Output:

```
Enter the passage to summarize:
 Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks that typically require human intelligence. It is widely used in healthcare for disease prediction, in finance for fraud detection, and in education for personalized learning. AI systems use techniques such as machine learning and deep learning to analyze large amounts of data and make intelligent decisions. As technology advances, AI continues to improve efficiency, accuracy, and productivity across various sectors.

Summarized Text:
Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks that typically require human intelligence. It is widely used in healthcare for disease prediction, in finance for fraud detection, and in education for personalized learning.
```

Experiment-8

Install langchain, cohere (for key), langchain-community. Get the API key (By logging into Cohere and obtaining the Cohere key). Load a text document from your Google Drive. Create a prompt template to display the output in a particular manner.

```
pip install langchain
pip install langchain-community
pip install cohere
!pip install -U langchain-cohere
```

To create API Key follow the below steps:
Go to https://dashboard.cohere.com/

1. Login
2. Click **API Keys**
3. Click **Create API Key**
4. Copy the key

```
from langchain_core.prompts import PromptTemplate
from langchain_cohere import ChatCohere
import os

# Step 1: Set your Cohere API key
os.environ["COHERE_API_KEY"] = "paste the API key here"

# Step 2: Load text document
file_path = "Artificial_Intelligence.txt"

with open(file_path, "r", encoding="utf-8") as file:
    document_text = file.read()

print("File loaded successfully!")

# Step 3: Create Prompt Template
prompt_template = PromptTemplate(
    input_variables=["text"],
    template="""
    You are an AI assistant.
    Summarize the following text in simple and clear language.
    Keep the summary short and meaningful.

    Text:
    {text}
    """
)

# Step 4: Initialize Cohere Chat Model
llm = ChatCohere(model="command-r-08-2024")

# Step 5: Generate Output
response = llm.invoke(prompt_template.format(text=document_text))
```

```
print("\nSummary:\n")
print(response.content)
```

Output:

File loaded successfully!

Summary:

Artificial Intelligence (AI) is a field of computer science that aims to develop intelligent machines capable of performing complex tasks typically associated with human intelligence. These tasks involve learning, problem-solving, language understanding, image and speech recognition, and decision-making.

Experiment-9

Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia:

- **The founder of the Institution.**
- **When it was founded.**
- **The current branches in the institution.**
- **How many employees are working in it.**
- **A brief 4-line summary of the institution.**

Packages to Install

```
!pip install wikipedia pydantic

 import wikipedia
import re
from pydantic import BaseModel


class InstitutionInfo(BaseModel):
    founder: str
    founded_year: str
    branches: str
    employees: str
    summary: str


institution = input("Enter Institution Name: ")

try:
    # Disable auto suggestion to avoid wrong page names
    summary = wikipedia.summary(institution, sentences=4, auto_suggest=False)
    page = wikipedia.page(institution, auto_suggest=False).content

    # Extract founder
    founder_match = re.search(r"founded by ([A-Za-z\s]+)", page, re.IGNORECASE)
    founder = founder_match.group(1) if founder_match else "Not available"

    # Extract year
    year_match = re.search(r"\b(18|19|20)\d{2}\b", page)
    founded_year = year_match.group(0) if year_match else "Not available"

    branches = "Multiple campuses / departments"
```

```
    employees = "Information available on Wikipedia"

    result = InstitutionInfo(
        founder=founder,
        founded_year=founded_year,
        branches=branches,
        employees=employees,
        summary=summary
    )

    print("\nInstitution Details\n")
    print("Founder:", result.founder)
    print("Founded Year:", result.founded_year)
    print("Branches:", result.branches)
    print("Employees:", result.employees)
    print("Summary:", result.summary)

except wikipedia.exceptions.PageError:
    print("The page was not found. Please enter a valid institution name.")

except wikipedia.exceptions.DisambiguationError as e:
    print("Multiple results found. Try one of these:")
    print(e.options[:5])
```

Experiment-10

Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.

```
!pip install sentence-transformers faiss-cpu langchain-community langchain-text-splitters pypdf
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
import re
import os

!pip install pypdf
import sys
!{sys.executable} -m pip install pypdf

import pypdf
print("Working ")

pdf_path = "ipc.pdf"   # make sure file is in same directory

loader = PyPDFLoader("C:/Users/Staff/Desktop/JN/ipc.pdf")
docs = loader.load()

print("Pages loaded:", len(docs))

splitter = CharacterTextSplitter(chunk_size=1200, chunk_overlap=200)
documents = splitter.split_documents(docs)

print("Total chunks created:", len(documents))

import sys
!{sys.executable} -m pip install sentence-transformers

import sentence_transformers
print("Installed ")

embeddings = HuggingFaceEmbeddings(
   model_name="sentence-transformers/all-MiniLM-L6-v2"
)

import sys
!{sys.executable} -m pip install faiss-cpu

db = FAISS.from_documents(documents, embeddings)
```

```
retriever = db.as_retriever(search_kwargs={"k": 5})

print("IPC Chatbot Ready!")

full_text = "\n".join([doc.page_content for doc in documents])


print("\nType 'exit' to stop.\n")

while True:
    query = input("Ask about IPC: ")

    if query.lower() == "exit":
        print("Chatbot stopped.")
        break

    match = re.search(r"\b\d+\b", query)

    if match:
        section_number = match.group()
        print(f"\n Extracting Section {section_number}...\n")

        # Extract section using regex (captures full section content)
        pattern = re.compile(
            rf"{section_number}\..*?(?=\n\d+\.)",
            re.DOTALL
        )

        result = pattern.search(full_text)

        if result:
            section_text = result.group()
            print(" Section Content:\n")
            print(section_text[:1000])  # limit output
        else:
            print(" Section not found, using fallback search...\n")
            results = retriever.invoke(query)

            for i, doc in enumerate(results[:3]):
                print(f"Result {i+1}:\n")
                print(doc.page_content[:500])
                print("\n----------------------------\n")

    else:
        results = retriever.invoke(query)

        print("\n Top Relevant IPC Content:\n")
```

```
for i, doc in enumerate(results[:3]):
    print(f"Result {i+1}:\n")
    print(doc.page_content[:500])
    print("\n-----------------------------\n")
```

Course outcomes (Course Skill Set):
At the end of the course the student will be able to:

- Develop the ability to explore and analyze word embeddings, perform vector arithmetic to investigate word relationships, visualize embeddings using dimensionality reduction techniques.
- Apply prompt engineering skills to real-world scenarios, such as information retrieval, text generation.
- Utilize pre-trained Hugging Face models for real-world applications, including sentiment analysis and text summarization.
- Apply different architectures used in large language models, such as transformers, and understand their advantages and limitations.

Suggested Learning Resources:

Books:

1. Modern Generative AI with ChatGPT and OpenAI Models: Leverage the Capabilities of OpenAI's LLM for Productivity and Innovation with GPT3 and GPT4, by Valentina Alto, Packt Publishing Ltd, 2023.
2. Generative AI for Cloud Solutions: Architect modern AI LLMs in secure, scalable, and ethical cloud environments, by Paul Singh, Anurag Karupati, Packt Publishing Ltd, 2024.

Web links and Video Lectures (e-Resources):

• https://www.w3schools.com/gen_ai/index.php
• https://youtu.be/eTPIiL3DF27U
• https://youtu.be/je6AlVeGOV0
• https://youtu.be/RLYqsA8ns6k
• https://youtu.be/OSAKM7wiC-A
• https://youtu.be/28_9xMyrdjg
• https://youtu.be/8iuiz-c-EBw
• https://youtu.be/7oQ8VtEKcgE
• https://youtu.be/seXp0VWWZv0